Solution: Next Permutation

Problem Statement

Given an array of integers `nums`, return the next permutation of `nums.`

A `permutation` of the array is an `arrangement` of its elements in the `linear order` or `particular sequence`.

- For example, permutations of the `[2, 1, 3]` array are `[3,2,1], [3,1,2], [1,2,3], [1,3,2], [2, 1, 3]`, and `[2, 3, 1]`.

A `next permutation` of the array is the `next smallest lexicographically greater` permutation of array elements.

- For example, next permutation of `[2, 1, 3]` is `[2, 3, 1]`, which is smallest lexicographically greater permutation.

If `no higher permutation` is possible, the array should be rearranged into its `lowest possible order` (ascending).

- For example, next permutation for `[3, 2, 1]` is `[1, 2, 3]` as `[3, 2, 1]` doesn't have lexicographically larger permutation.

Examples

- **Example 1**:
  - **Input**: `[1, 3, 5, 4, 2]`
  - **Expected Output**: `[1, 4, 2, 3, 5]`
  - **Justification**: The next permutation is obtained by swapping `3` with the next higher number (`4`) and then sorting the numbers after `4` in ascending order.
- **Example 2**:
  - **Input**: `[2, 3, 6, 5, 4, 1]`
  - **Expected Output**: `[2, 4, 1, 3, 5, 6]`
  - **Justification**: The next permutation is found by swapping `3` with the next higher number (`4`) in the sequence following it and then sorting the numbers after `4` in ascending order.
- **Example 3**:
  - **Input**: `[1, 1, 5]`
  - **Expected Output**: `[1, 5, 1]`
  - **Justification**: The next immediate permutation after `[1, 1, 5]` is `[1, 5, 1]`.

Solution

To solve this problem, the key idea is to modify the sequence slightly to obtain the next greater arrangement of numbers. This approach can be visualized as follows: starting from the right end of the array, we look for the first pair of consecutive numbers where the left number is smaller than the right one. This left number is termed as the 'pivot'. The pivot is the number that we need to increase to get the next higher permutation. However, to ensure the smallest increase and maintain the order of a valid next permutation, we swap the pivot with the smallest number that is greater than it among the numbers to its right. After the swap, the sequence to the right of the original pivot position is reversed to ensure we get the lowest possible sequence there, completing the formation of the next permutation. This approach ensures we find the next permutation in an efficient and methodical way, without the need for generating all permutations.

This method is effective as it leverages the properties of permutations - an ordered arrangement of numbers. By making minimal changes - a single swap and a reversal, we move directly to the next sequence that is numerically just above the current one. This approach is efficient as it operates in linear time complexity, avoiding any unnecessary computations.

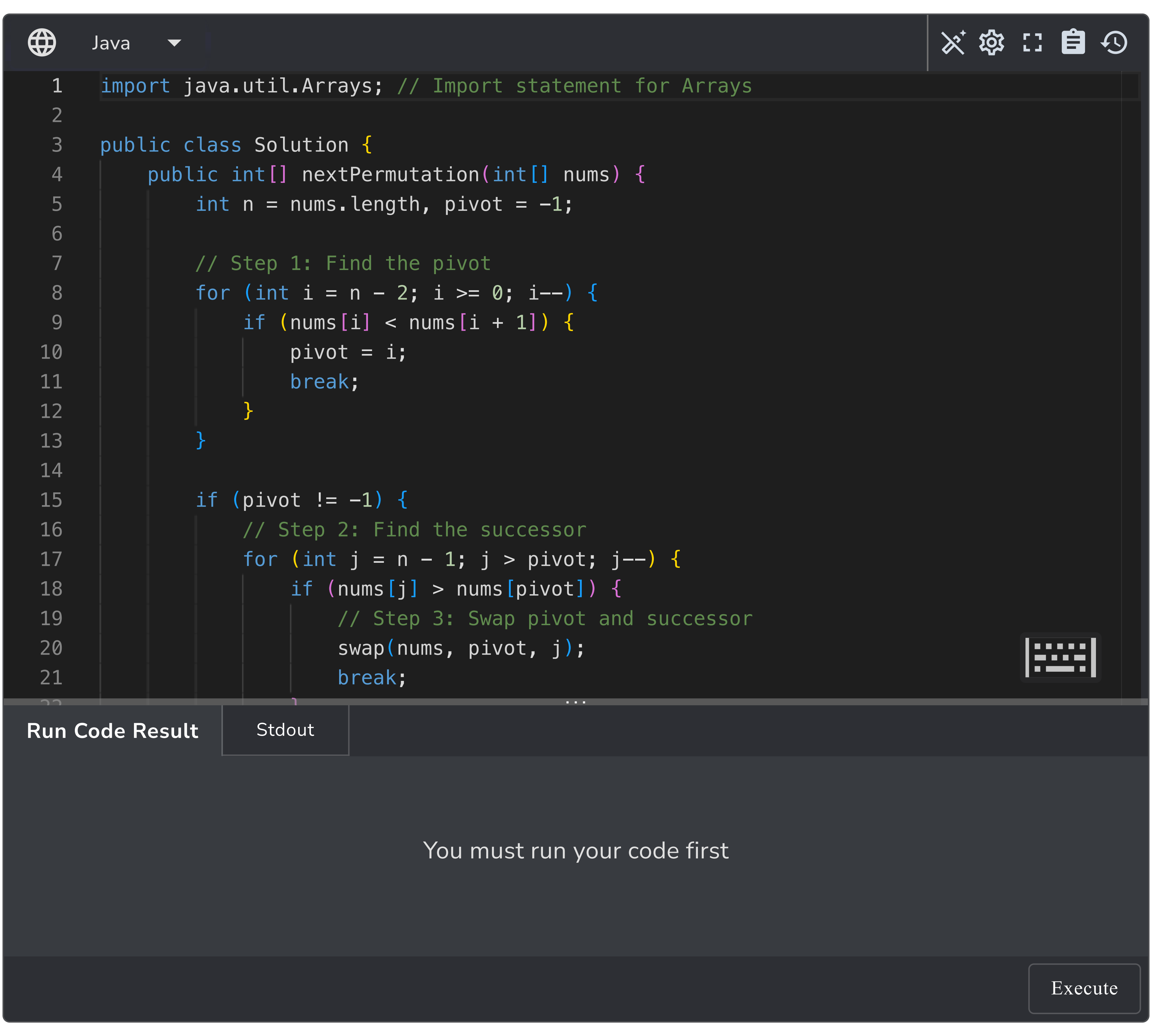
Step-by-step Algorithm

1. **Initialize Variables**:
   - Let `n` be the length of the input array `nums`.
   - Initialize `pivot` to -1. The `pivot` variable will store the index of the pivot element we find.
2. **Find the Pivot**:
   - Iterate backwards from the second last element (index `n - 2`) to the start of the array.
   - If an element `nums[i]` is found such that `nums[i] < nums[i + 1]`, set `pivot = i`. Break the loop.
3. **Find the Successor (if Pivot is Found)**:
   - If `pivot` is not -1 (pivot exists), iterate backwards from the last element (index `n - 1`) to find the successor.
   - Find the smallest element `nums[j]` such that `nums[j] > nums[pivot]`.
4. **Swap Pivot and Successor**:
   - If a pivot was found (`pivot` != -1), swap the elements `nums[pivot]` and `nums[j]`.
5. **Reverse the Sequence after Pivot Position**:
   - Reverse the sub-array of `nums` starting from `pivot + 1` to the end of the array.
   - This can be done by swapping elements symmetrically around the midpoint of the sub-array from `pivot + 1` to `n - 1`.

Algorithm Walkthrough

- Take the input `[2, 3, 6, 5, 4, 1]`.
- Find the pivot: The rightmost digit smaller than its right neighbor is `3` (since `3` is smaller than `6`).
- Find the successor: The smallest digit larger than `3` in the sequence to its right is `4`.
- Swap `3` and `4`, the array becomes `[2, 4, 6, 5, 3, 1]`.
- Reverse the sequence after `4`, which is `[6, 5, 3, 1]`, to get `[1, 3, 5, 6]`.
- The array after reversal is `[2, 4, 1, 3, 5, 6]`.
- Final result is `[2, 4, 1, 3, 5, 6]`.

Code



```java
import java.util.Arrays; // Import statement for Arrays

public class Solution {
    public int[] nextPermutation(int[] nums) {
        int n = nums.length, pivot = -1;

        // Step 1: Find the pivot
        for (int i = n - 2; i >= 0; i--) {
            if (nums[i] < nums[i + 1]) {
                pivot = i;
                break;
            }
        }

        if (pivot != -1) {
            // Step 2: Find the successor
            for (int j = n - 1; j > pivot; j--) {
                if (nums[j] > nums[pivot]) {
                    // Step 3: Swap pivot and successor
                    swap(nums, pivot, j);
                    break;
```

You must run your code first

Complexity Analysis

- **Time Complexity**: The algorithm runs in `O(N)` time, where `N` is the number of elements in the array. This efficiency is due to linear scans to find the pivot and successor, and a single reverse operation.
- **Space Complexity**: `O(1)`, as the rearrangement is done in-place with no need for extra space.

← Previous
Next Permutation (Medium)
Next →
Simplify Path (Medium)
☐ Mark as Completed